

GUÍA PARA CARGAR UN PROGRAMA Y EJECUTARLO EN LA FPGA.

En esta guía ya asumimos que el lector tiene instalado tanto Quartus Lite (Ver. 20.1.1) y el compilador RISC-V GCC.

Se va a asumir también que este repo va a estar instalado a la hora de ejecutar la tabla.

Generar archivos de inicialización de memoria.

Este paso es importante porque estos archivos nos permiten después generar el archivo .sof en Quartus con el que vamos a programar la placa. `$ cd software/ $ python3 export_opcode_rv.py ../components c_progs/csum.c` Se puede reemplazar el archivo csum.c por cualquier archivo .c que quieran usar para probar.

Compilar en Quartus usando los archivos de inicialización de memoria

Para este caso, vamos a tener que abrir Quartus y abrir el proyecto usando el archivo `./de0_nano/DE0_NANO.qpf` para generar un archivo .sof, este va a ser el mismo que vamos a utilizar para programar la FPGA.

Programamos la placa FPGA.

Ya con estos pasos resueltos, nos vamos a esta dirección desde donde estamos:

```
$ cd ../de0_nano/nios/software/hw_io/
```

Ahí mismo, inicializamos la shell de nios2:

```
$ QUARTUS_INSTALL/nios2eds/nios2_command_shell.sh
```

Ahora conectamos la FPGA, y ejecutamos el script de `fault_injection.py` para asegurarnos que cada uno de los pasos funcionen de la siguiente forma:

```
python3 fault_injection.py
```

PENDIENTE:

Guía detallada para la ejecución haciendo lo mismo que hace `fault_injection.py` pero paso a paso a mano, así como también una guía para la interpretación del output.

Posibles fallas a la hora de intentar ejecutar

A la hora de ejecutar, no se reconoce la FPGA.

Este caso se soluciona modificando el este archivo de la siguiente forma:

```
$ sudo nano /etc/udev/rules.d/51-altera-usb-blaster.rules
```

Y dentro de ese mismo archivo, agregamos el siguiente fragmento:

```
# USB-Blaster
SUBSYSTEM=="usb", ATTRS{idVendor}=="09fb", ATTRS{idProduct}=="6001",
MODE="0666"
SUBSYSTEM=="usb", ATTRS{idVendor}=="09fb", ATTRS{idProduct}=="6002",
MODE="0666"
SUBSYSTEM=="usb", ATTRS{idVendor}=="09fb", ATTRS{idProduct}=="6003",
MODE="0666"
# USB-Blaster II
SUBSYSTEM=="usb", ATTRS{idVendor}=="09fb", ATTRS{idProduct}=="6010",
MODE="0666"
SUBSYSTEM=="usb", ATTRS{idVendor}=="09fb", ATTRS{idProduct}=="6810",
MODE="0666"
```

No me reconoce los comandos quartus_pgm o nios2-terminal

Para el primer caso, una resolución es cambiar las referencias que tenés dentro del código al camino completo de la ubicación del archivo. Como referencia, así estaba el archivo programmer_helper.sh:

```
CWD=$(pwd)

quartus_pgm -m jtag -o "p;DE0_NANO.sof"
```

Esto fallaba por no encontrar una referencia a quartus_pgm, se puede solucionar de la siguiente manera:

```
CWD=$(pwd)

~/intelFPGA_lite/20.1/quartus/bin/quartus_pgm -m jtag -o "p;DE0_NANO.sof"
```

Para el segundo caso, con nios2-terminal, antes de intentar cambiar la referencia por el path completo, hay que verificar que estemos ejecutando el entorno de nios2 antes. Explicado en el paso "Programamos la placa FPGA"

No me reconoce el compilador de gcc para RISC-V a la hora de ejecutar export_opcode.py

De la misma forma que en el problema anterior, una solución muy sencilla a realizar es cambiar cualquier referencia que tenga el compilador, por el path completo donde lo tenemos instalado.

Esto se puede hacer cambiando en los archivos `export_opcode.py` y `rv_binary_patcher` la variable `TOOLCHAIN` de la siguiente forma:

Antes:

```
TOOLCHAIN = "riscv64-unknown-elf"
```

Después:

```
TOOLCHAIN = "/opt/riscv/bin/riscv64-unknown-elf"
```

Asumiendo que instalaron el compilador en la dirección por default.